

Практическое задание №5. Обработка одномерных массивов.

На оценку 4 делаем задание на 4. На оценку 5 необходимо выполнить на оценку 4 и затем один из вариантов на оценку 5.

На оценку 4:

В современных системах связи (Wi-Fi, мобильные сети 4G/5G) сигналы обычно представляются в виде комплексных чисел. Это позволяет компактно описывать амплитуду и фазу сигнала. Например, в модуляции QAM (Quadrature Amplitude Modulation) каждое комплексное число соответствует определенному символу, передающему несколько бит информации.

Комплексное число $z = a + bj$, где z – комплексное число, состоящее из a (реальная часть) и b (мнимая) – некоторых целых/вещественных чисел, а $j = \sqrt{-1}$. На практике такое комплексное число можно представить из двух отдельных переменных. Массив же из комплексных чисел есть массив чисел с реальной и мнимой частью, где, например, числа под четными индексами — это реальная часть комплексного числа, а под нечетными мнимая часть.

Например, массив из 3 комплексных чисел – есть массив из 6 обычных чисел: [RE0, IM0, RE1, IM1, RE2, IM2], RE – Real (реальная часть), IM – Imag (imaginary, мнимая часть), а число – индекс комплексного числа.

Задача:

1. Объявить массив размером N для хранения в нем **комплексных чисел**
2. **Заполнить любым способом любыми числами** в диапазоне [-32768..32767]. Возможные варианты: случайные числа, ввод с клавиатуры или инициализация чисел прям в коде
3. Вывести на экран значения сигнала в виде $s[i] = re + j*(im)$, пример:
 $s[0] = -1234.0 + j*(-3000.0)$
 $s[1] = 1000.0 + j*(1000.0)$
 $s[2] = -3.0 + j*(-1.0)$
4. Вывести на экран чему равна **энергия сигнала**. Энергия сигнала вычисляется по формуле

$$signal_energy = \sum_{k=0}^n re[k] * re[k] + im[k] * im[k]$$

5. Найти и вывести на экран **максимальный элемент** среди **реальной части**.

На оценку 5:

Реализовать программу согласно вашему варианту. Заполнить массивы любым способом. Разрешается взять ± 1 вариант от своего, например для 7 варианта можно взять и решить один из трех вариантов 6, 7, 8.

Требование для всех вариантов: Обязательно используем функции для разделения программы на логические блоки (подзадачи), например, возможные функции – вывод на экран массива, определение простое ли число, поиск максимального, вычисление среднего и т.д..

Варианты заданий

1. Записать каждый второй элемент целочисленного массива $X=(x_1, x_2, \dots, x_n)$ подряд в массив $Y=(y_1, y_2, \dots, y_k)$. Определить количество простых чисел в каждом массиве. Вычислить среднее арифметическое всех элементов массивов X и Y .
2. Определить максимальный элемент среди положительных нечетных элементов и минимальный среди положительных четных элементов целочисленного массива $X=(x_1, x_2, \dots, x_n)$. Заменить на ноль все совершенные числа, вывести сообщение, сколько элементов было обнулено.
3. В массиве $X(n)$ после каждого отрицательного элемента в последующий элемент вставить ноль. Определить, поменялось ли местоположение минимального элемента массива. Найти сумму четных и произведение нечетных элементов массива.
4. Определить, содержит ли заданный массив группы элементов, расположенные в порядке возрастания их значений. Если да, то определить количество таких групп. Обнулить первую такую группу.
5. В массиве $X=(x_1, x_2, \dots, x_n)$ определить количество элементов, меньших среднего арифметического значения. Обнулить элементы, расположенные между максимальным и минимальным.
6. В заданном массиве целых чисел найти самую маленькую серию подряд стоящих нечетных элементов. Заменить на ноль два первых простых числа. Проверить, изменилась ли серия подряд стоящих нечетных элементов.
7. Вычислить среднее арифметическое элементов массива $X=(x_1, x_2, \dots, x_n)$, расположенных между его минимальным и максимальным значениями.
8. Определить порядковые номера и значения первого положительного и последнего отрицательного элементов целочисленного массива $X(n)$. Определить среднее арифметическое элементов массива, позиционно

расположенных между найденными элементами. Предусмотреть случай, что массив может не содержать положительных или отрицательных элементов.

9. Заменить на ноль в массиве целых чисел все двузначные элементы, являющиеся простыми числами. Найти среднее арифметическое элементов массива до и после удаления. Проверить, изменился ли максимальный элемент массива.

10. Преобразовать заданный массив целых положительных чисел $F(n)$ таким образом, чтобы цифры каждого его элемента были записаны в обратном порядке. Определить количество простых чисел в массиве до и после преобразования.

11. Задан массив $Y(k)$ целых чисел. Если он упорядочен (по возрастанию или убыванию), то оставить его без изменения. Если массив не упорядоченный, то заменить каждый четный элемент на минимальное непростое число в массиве.

12. Задан массив $X(m)$ целых чисел. Поменять местами в массиве последнее простое число и первое совершенное. Предусмотреть случай, что массив может не содержать простых и совершенных чисел.

13. Задан массив $Y(k)$ целых чисел. Определить в массиве количество простых двузначных чисел. Если таких чисел больше двух, обнулить все такие числа. Проверить, изменился ли максимальный элемент массива.

14. Задан массив $X(n)$ целых чисел. Заменить на ноль в массиве все элементы, большие среднего арифметического значения. Определить в массиве количество простых и совершенных чисел до и после обнуления.

15. Задан массив X целых чисел. Если массив не является знакопередающим, то присвоить нулю в массиве все положительные числа, в противном случае – присвоить нулю отрицательные элементы. После обнуления определить количество нечетных чисел.

16. Задан массив $Z(m)$ целых чисел. Определить, содержит ли массив серии из подряд стоящих простых чисел. Если да, то посчитать количество таких серий. Присвоить нулю последнюю такую серию.

Под Звездочкой (+ балл):

Реализовать алгоритм сжатия данных **RLE (Run-Length Encoding)** для одномерного массива. Алгоритм RLE заменяет повторяющиеся последовательности одинаковых элементов на пары (значение, счетчик). Для

проверки также написать алгоритм восстановления сжатых данных в исходный массив. Пример работы:

```
Исходный: [1,1,1,1,2,2,3,4,4,4,4,4,5,0,0,0,0,0,0,0]
Сжатый:   [1,4, 2,2, 3,1, 4,5, 5,1, 0,7]
Восстановленный: [1,1,1,1,2,2,3,4,4,4,4,4,5,0,0,0,0,0,0,0]
Коэффициент сжатия: 40.0% (исходный:20 элементов, сжатый:12 элементов)
```

Прототипы ф-ий, которые можно взять за основу:

```
int rle_compress(int input[], int size, int compressed[]) {
    int compressed_size; // размер после сжатия
    // реализация здесь

    return compressed_size;
}

int rle_decompress(int compressed[], int compressed_size, int output[], int max_output_size) {
    int output_size; // размер после восстановления
    // реализация здесь

    return output_size;
}

void print_compressed(int compressed[], int size) {
    // реализация здесь
}
```

Тестовые данные:

Тест 1: Все элементы одинаковые

```
int test1[] = {5,5,5,5,5,5,5,5,5,5};
```

Ответ: [5, 10]

Тест 2: Все элементы разные

```
int test2[] = {1,2,3,4,5,6,7,8,9,10};
```

Ответ: [1,1, 2,1, 3,1, 4,1, 5,1, 6,1, 7,1, 8,1, 9,1, 10,1]

Тест 3: Чередующиеся последовательности

```
int test3[] = {1,1,2,2,1,1,2,2};
```

Ответ: [1,1, 2,1, 1,1, 2,1]